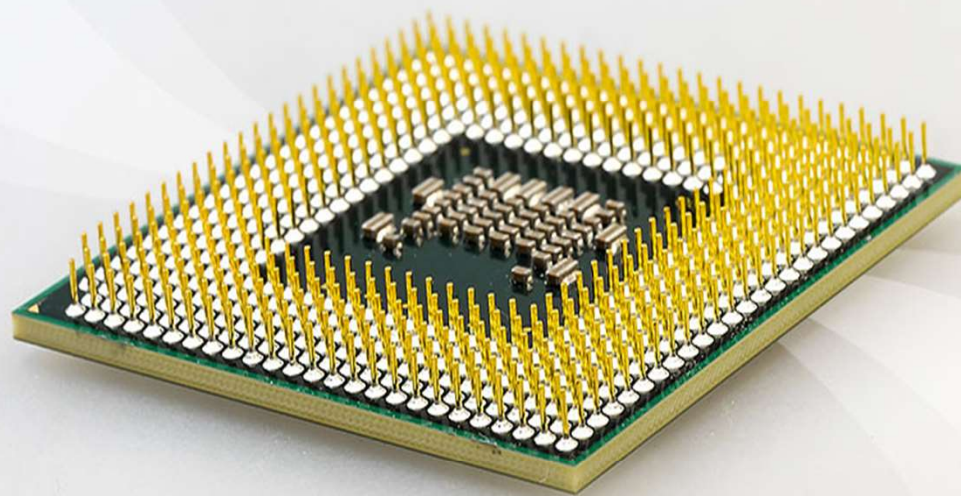


Đại Học Công Nghiệp TP HCM



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL

GV: Vũ Thị Hồng Nga

Biên soạn năm 2019

Chương 3: Ngôn ngữ mô tả phần cứng Verilog HDL

I. Giới thiệu

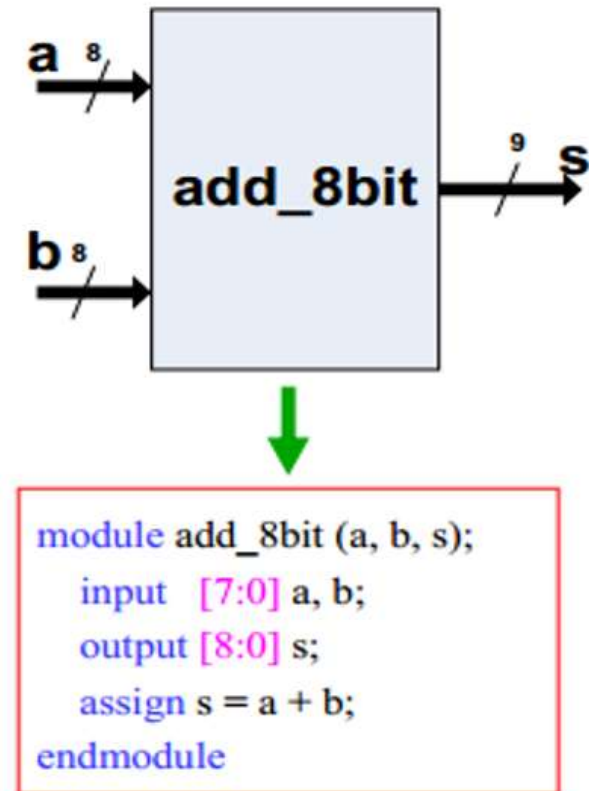
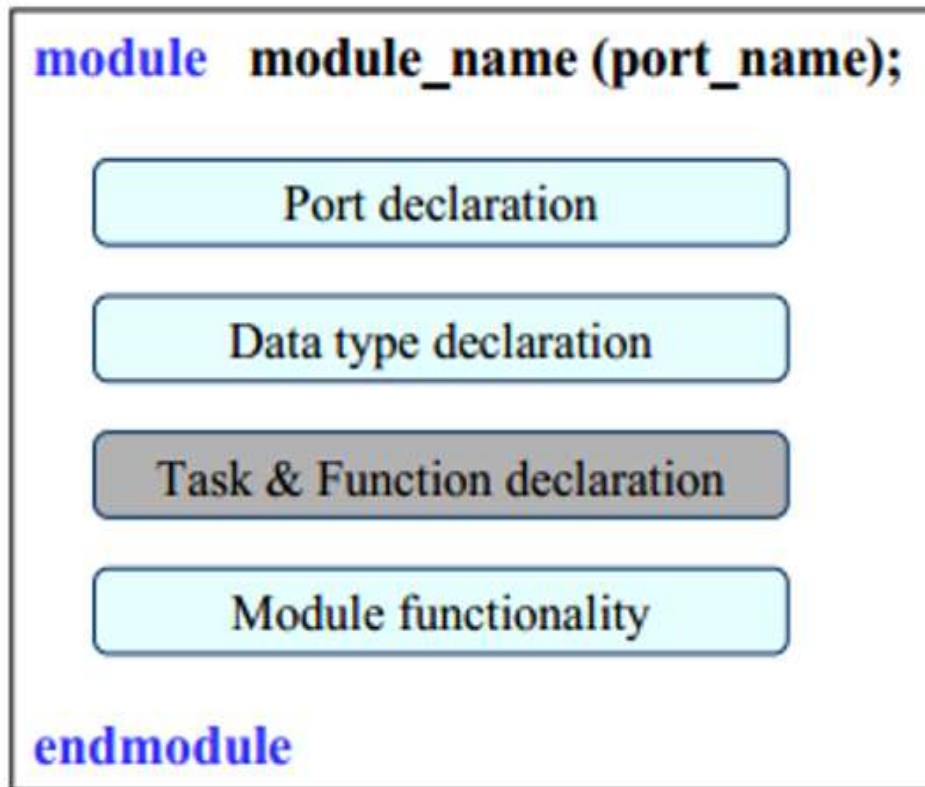
- Verilog là ngôn ngữ mô tả phần cứng, được sử dụng trong việc thiết kế các các hệ thống số, các mạch tích hợp...
- Verilog lần đầu được giới thiệu vào năm 1984 bởi công ty Gateway Design Automatic. Năm 1995 Verilog chính thức được chuẩn hóa bởi tổ chức IEEE.
- Verilog được dùng để xây dựng các ứng dụng trên nền các công nghệ như FPGA, CPLDs...

1. Các mức mô tả trong Verilog

- Algorithmic level : sử dụng các phát biểu if, case and loop statements..
- Register transfer level (RTL) : sử dụng các thanh ghi liên kết các biểu thức boolean
- Gate level : kết nối mạch dưới dạng các cổng logic AND, NOR....
- Switch level : kết nối mạch dưới dạng các MOS transistors bên trong các cổng logic AND, NOR....

.

2. Cấu trúc một Module



II. Ngôn ngữ mô tả phần cứng Verilog HDL

1 Các cú pháp trong verilog

➤ **Khoảng trắng:**

dùng để ngăn các từ khóa và có thể chứa khoảng cách, khoảng dài, dòng mới và dạng đường dẫn.

➤ **Chú thích:**

- Chú thích được viết sau hai dấu gạch xiên (//) khi viết trên một dòng.
- Chú thích được viết giữa /* */, khi viết trên nhiều dòng

Ví dụ:

```
a = c + d; // cộng c và d
```

```
/* đây là phép cộng hai số nhị phân c và d, kết quả được gán vào a */
```

➤ Các giá trị của biến trong verilog

0 : mức logic 0, hoặc điều kiện sai.

1 : mức logic 1, hoặc điều kiện đúng.

X : mức logic tùy định

Z : trạng thái tổng trở cao

➤ Biểu diễn số trong verilog:

`<kích thước> '<hệ số> <giá trị số>`

Kích thước: số bit chứa giá trị số

Hệ số:

Số nhị phân → B hoặc b

Số thập phân → D hoặc d

Số thập lục phân → H hoặc h

Số bát phân → O hoặc o

Ví dụ:

4'b1011 → số nhị phân 1011 lưu trong 4 bit

8'hEF → số thập lục phân EF lưu trong 8 bit

16'o56 → số bát phân 56 lưu trong 16 bit

4'bxxxx → số nhị phân tùy định lưu trong 4 bit

4'bzzzz → số nhị phân có giá trị tổng trở cao lưu
trong 4 bit

Chú ý: nếu không ghi hệ số thì xem như số đó được biểu diễn ở dạng thập phân

➤ Chuỗi:

- Chuỗi là một dãy các kí tự được nằm trong hai dấu nháy kép (“ ”) và được ghi trên một dòng đơn.
- Trong những phép gán, chuỗi được xem như là những hằng số nguyên không dấu và được biểu diễn bởi một dãy kí tự 8 bit ASCII

Ví dụ:

chuỗi 13 kí tự trên 1 dòng, mỗi ký tự mã hóa bằng 8 bit

```
Reg [8*13:1] String_val;
```

```
String_val = “verilog HDL”;
```

➤ Từ định danh:

- Từ định danh do người dùng quy định cho biến số, tên hàm, tên môđun, tên khối và tên trường hợp
- Từ định danh bắt đầu bằng một ký tự hoặc đường gạch dưới '_' nhưng không được bắt đầu bằng một con số hoặc \$
- Từ định danh không được đặt trùng với các từ khóa của Verilog như: module, write, register....
- Verilog phân biệt chữ hoa và chữ thường

➤ **Biến Wire:**

- Hoạt động như một dây nối vật lý kết nối giữa các linh kiện
- Giá trị của Wire có thể đọc, nhưng không được gán trong hàm (function) hoặc khối (block)
- Các dạng khác của biến Wire là : wand, wor, tri
- Biến wire không lưu giữ giá trị của nó mà được điều khiển bằng một trong hai cách sau:
 - + Kết nối wire đến ngõ ra của một cổng hay một module
 - + Gán một giá trị đến wire trong một phép gán nối tiếp

Cú pháp của Wire:

```
Wire  wire_variable_list;  
Wire  [msb:lsb] wire_variable_list;
```

Ví dụ:

```
Wire      a;    // a là biến wire 1 bit  
Wire [0:7] b;    // b là biến wire 8 bit, b[0] là MSB  
Wire [3:0] c;    // c là biến wire 4 bit, c[3] là MSB
```

➤ Biến reg (register):

- Biến reg hoạt động giống như biến trong các ngôn ngữ lập trình cấp cao
- reg lưu trữ giá trị của phép gán cuối cùng cho đến khi nó được gán một giá trị mới trong mô tả hành vi, khối initial hoặc always.

Cú pháp của reg:

```
reg    reg_variable_list;  
reg    [msb: lsb] reg_variable_list;
```

Lưu ý: khai báo reg được dùng cho các tín hiệu trong mô tả hành vi

➤ **Tham số (parameter):** là hằng số có giá trị xác định được sử dụng trong module

Ví dụ: bộ cộng nhị phân 8 bit

```
module adder (a,b,s);  
parameter n = 7;  
input [n:0] a,b;  
output [n+1:0] s;  
    assign s = a + b;  
endmodule
```

➤ **Input, Output, Inout:**

là ngõ vào, ngõ ra, và port hai chiều của một module hoặc task. Một port ngõ ra có thể được cấu hình từ các dạng: wire, reg, wand, wor, hoặc tri nhưng mặc định là wire.

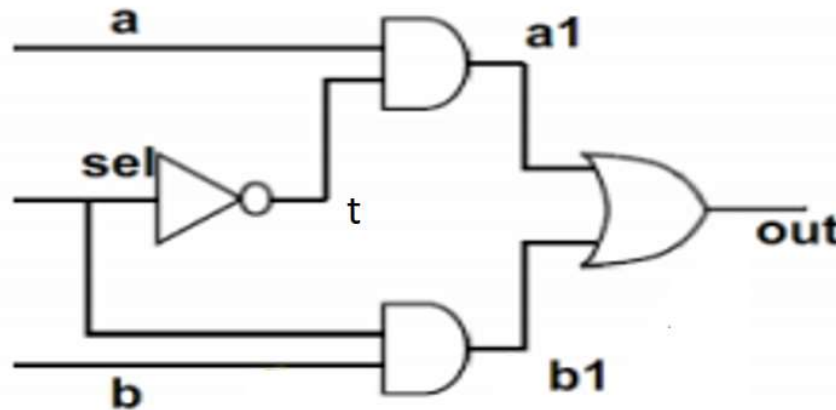
```
input  [msb:lsb]  input_port_list;  
output [msb:lsb]  output_port_list;  
inout  [msb:lsb]  inout_port_list;
```

Ví dụ:

Input [3:0] a; // a là input 4 bit, a[3] là MSB

Output [0:7] b // b là output 8 bit, b[0] là MSB

Ví dụ: khai báo biến cho mạch tổ hợp sau



```
module machtohop (a, b, sel, out);  
    input a,b,sel;  
    output out;  
    wire t, a1, b1;  
    :  
endmodule
```

➤ Số nguyên và số thực

- Là biến đa năng, trong tổng hợp chúng được dùng chủ yếu cho vòng lặp, tham số, và hằng số...
- Giá trị của biến là số nguyên hoặc số thực mặc định được lưu vào thanh ghi 32 bit

2. Các toán tử trong Verilog

a. Toán tử số học: Là các toán tử thực hiện các phép toán số học trong Verilog

+	Phép cộng
-	Phép trừ
*	Phép nhân
/	Phép chia lấy phần nguyên
%	Phép chia lấy phần dư

b. Toán tử bit-wise:

Thực hiện các phép not, or, and, xor, nxor trên từng cặp bit của các toán hạng

~	bitwise not
&	bitwise and
	bitwise or
^	bitwise xor
~^ / ^~	bitwise xnor

Ví dụ: cho $a = 1011$; $b = 1100$

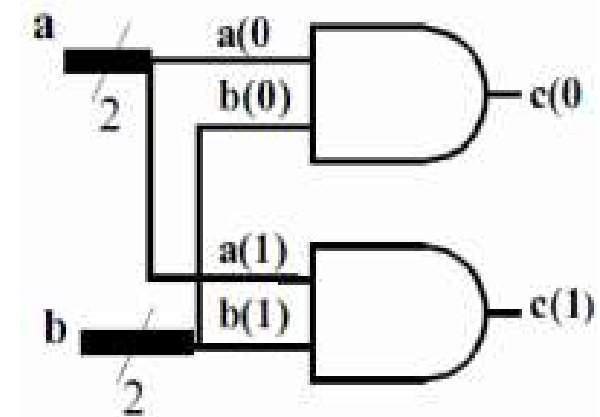
$c = a \& b = 1000$;

$d = a \wedge b = 0111$;

$e = \sim a = 0100$;

Ví dụ: mạch logic sử dụng toán tử bitwise and

```
module and2 (a, b, c)  
input  [1:0] a, b;  
output [1:0] c;  
assign c = a & b;  
endmodule
```



c.Toán tử quan hệ: Thực hiện so sánh hai toán hạng và trả về kết quả là 0 hoặc 1

<	nhỏ hơn
<=	nhỏ hơn hoặc bằng
>	Lớn hơn
>=	lớn hơn hoặc bằng
==	bằng
!=	khác (không bằng)

Ví dụ: cho $X = 4'b1001$, $Y = 4'b 1010$, tính giá trị của F

```
module so_sanh (X,Y,F);  
  input  [3:0] X,Y;  
  output [3:0] F;  
  reg    [3:0] F;  
  always @ (X or Y)  
    if (X>Y)  
      F = X & Y;  
    else  
      F = X | Y;  
endmodule
```

d. Toán tử logic:

- Toán tử logic trả về một trong các giá trị: 0, 1, x.
- Chúng có thể làm việc trên biểu thức, số nguyên hoặc nhóm bit, và coi tất cả các giá trị khác '0' là '1'
- Toán tử logic được dùng nhiều trong lệnh điều kiện

&&	and logic
	or logic
!	Not logic

VD: cho $a = 4'b1001$, $b = 4'b1010$, $s1 = 1$, $s2 = 0$.
Tính giá trị của F

```
module vidu (a,b,s1,s2,f);  
input [3:0]a,b;  
input s1,s2;  
output reg [3:0]f;  
always @(a or b or s1 or s2)  
if ((s1 == 1'b1) &&(s2 == 1'b1))  
    f = a^b;  
else  
    f = ~b;  
endmodule
```

e. Toán tử giảm:

Thao tác trên tất cả các bit của một vectơ toán hạng và trả về giá trị đơn bit. Những toán tử này là hình thức tự đổi số của các toán tử bit-wire

&	reduction and
	reduction or
~&	reduction nand
~	reduction nor
^	reduction xor
~^ / ^~	reduction xnor

VD: $X = 4'b1010$

$A = \&X \quad \rightarrow \quad A = 1\&0\&1\&0 = 1'b0$

$C = |X \quad \rightarrow \quad C = 1|0|1|0 = 1'b1$

$D = ^X \quad \rightarrow \quad D = 1^0^1^0 = 1'b0$

f. Toán tử dịch:

Thực hiện dịch trái hoặc dịch phải n vị trí như trong cú pháp dưới đây, các bit ở vị trí trống sẽ được điền vào bằng số 0.

<<	dịch trái
>>	dịch phải

Ví dụ:

$A = 4'B1110;$

$D = A >> 2; \quad \rightarrow \quad D = 4'B0011$

$C = A << 1; \quad \rightarrow \quad C = 4'B1100$

g. Toán tử ghép:

Thực hiện ghép hai hoặc nhiều toán hạng thành một vectơ lớn hơn

$\{ \text{item1}, \text{item 2}, \dots, \text{item n} \}$

Ví dụ:

cho $A = 1'B1$; $B = 2'B00$; $C = 2'B10$; $D = 3'B110$

$Y = \{B, C\}; \quad \rightarrow Y = 4'B0010$

$E = \{A, B, C, D, 3'B001\} \quad \rightarrow E = 11'B10010110001$

$F = \{A, B[0], C[1]\} \quad \rightarrow F = 3'B101$

h. toán tử thứ bản:

Tạo ra nhiều bản sao của một mục chọn.

$\{n\{item\}\}$

Ví dụ:

$A = 2'B10;$

$B = 3'B101;$

$T = \{3\{A\}\};$

$X = \{2\{1'B0\}, A\};$

$Y = \{3\{A\}, 2\{B\}\};$

$\rightarrow T = 6'B101010;$

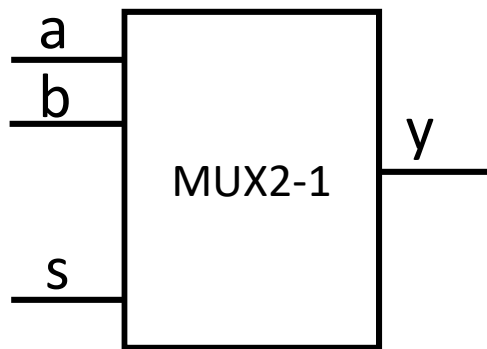
$\rightarrow X = 4'B0010$

$\rightarrow Y = 12'B101010101101$

i. Toán tử điều kiện:

Cond ? (Result if cond true) : (result if cond false)

Ví dụ: Mux 2 to 1



```
module mux2to1 (y,a,b,s);  
input a,b,s;  
output y;  
    assign y = s ? b : a;  
endmodule
```

j. Toán tử chọn bit:

Variable_name [index]
Variable_name [msb:lsb]

Ví dụ:

a = 8'b11110000;

b = 4'b1010;

d = a[7] & b[0];

→ d = 1'b0

c = a[3:0] | b[3:0];

→ c = 4'b1010

3. Phép gán liên tục

- Phép gán liên tục là phép gán sẽ được thực thi bất cứ khi nào giá trị của biểu thức bên phải thay đổi. Biểu thức bên trái phải có dữ liệu kiểu net.
- Cú pháp:

```
assign biến_wire = biểu_thức;
```

ví dụ:

```
input a,b;  
wire c;  
assign  c = a & b;
```

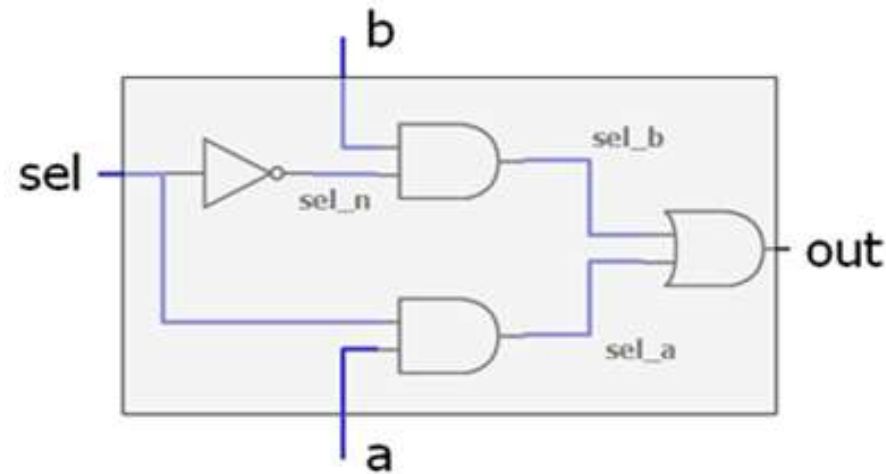
Các toán tử được sử dụng trong dataflow module

Operator Type	Operator Symbol	Operation Performed
Arithmetic	*, /, +, -	As expected
	%	Modulo
Logical	!	Logic NOT
	&&	Logic AND
		Logic OR
Bitwise	~	Bitwise NOT
	&	Bitwise AND
		Bitwise OR
	^	Bitwise XOR
	~^	Bitwise XNOR
Relational	<, >, <=, >=	As expected
Equality	==, !=	As expected
Reduction	&	Red. AND
	~&	Red. NAND
		Red. OR
	~	Red. NOR
	^	Red. XOR
	~^	Red. XNOR
Shift	<<	Left shift
	>>	Right shift
Concat	{ }	Concatenate
Replicate	{ { } }	Replicate
Cond	?:	As expected

Cấp độ ưu tiên của các toán tử trong dataflow module

Nử lý logic	Toán tử	Mức ưu tiên
Logic negation and Bit-wise negation (Lấy bù)	!	Cao ↑ Thấp
Multiply, divide operations (Nhân, chia)	~	
	*	
	/	
Plus, minus operations (Cộng, trừ)	%	
	+	
Shift operation (Dịch)	-	
	<<	
Comparison operation (So sánh hơn)	>>	
	< <= > >=	
Logical Equality, case equality (sà sánh bằng)	< <= > >=	
Bit-wise AND operation	== != === !==	
Bit-wise exclusive operation	&	
Bit-wise OR operation	^ ^~	
Logical AND operation		
Logical OR operation	&&	
Conditional expression (Điều kiện)		
	? :	

Ví dụ: mô tả mạch tổ hợp sau dùng dataflow model



```
module machtohop (out, a, b, sel);  
  input  a, b, sel;  
  output out;  
  assign out = (sel & a) | (~sel & b);  
endmodule
```

4. Phát biểu always

a. Cú pháp:

```
Always @ (event_expression) begin  
    <statements>  
end
```

@	: ký hiệu đặc tả sự kiện điều khiển
event_expression	: sự kiện điều khiển (biến nhảy)
Statements	: các phát biểu trong khối always

b. Chức năng của khối always

- Các phép gán trong always được sử dụng để cập nhật giá trị cho biến
- khi có bất kỳ biến nhảy nào thay đổi thì các phát biểu trong khối always được thực thi
- Phần biến bên trái của biểu thức trong khối always thông thường là : biến reg, integer, một phần trong một vector, một phần trong một mảng
- Phần gán bên phải là một biểu thức hoặc một function.

c. Các dạng của always:

always @ (a) : thực thi khi a thay đổi

always @ (a or b) : thực thi khi a hoặc b thay đổi

always @ (posedge ck) : thực thi khi có cạnh lên của ck

always @ (negedge ck) : thực thi khi có cạnh xuống của ck

Lưu ý:

- Các toán tử or có thể thay thế bởi dấu phẩy
- Khi tất cả các ngõ vào đều là sự kiện điều khiển thì ta có thể thay danh sách các biến nhạy bằng dấu *

Ví dụ: mô tả cổng and 3 ngõ vào a, b, c. một ngõ ra y

```
module machlogic (y,a,b,c);  
input a,b,c;  
output y;  
reg y;  
    always @ (a or b or c) begin  
        //always @ (a,b,c) begin  
        //always @ (*) begin  
            y = a & b & c;  
        end  
    endmodule
```

Ví dụ: mô tả HA dùng mô hình hành vi

```
module haft_adder(s, c, a, b);  
    input  a, b;  
    output s, c;  
    reg    s, c;  
    always @(a or b) begin  
        s = a ^ b;  
        c = a & b;  
    end  
endmodule
```

Ví dụ: mô tả D_FF dùng mô hình hành vi

```
module dff (q, d, clk);  
output q;  
input d, clk;  
reg q  
always @ (posedge clk) begin  
    q <= d;  
end  
endmodule
```

5. phát biểu initial

Phát biểu initial dùng để khởi tạo giá trị ban đầu cho các biến và chỉ được thực thi 1 lần vào thời điểm bắt đầu của quá trình mô phỏng

Ví dụ:

```
reg x, y, z;  
initial begin // complex statement  
    x = 1'b0; y = 1'b1; z = 1'b0;  
#10    x = 1'b1; y = 1'b1; z = 1'b1;  
end  
initial x = 1'b0; // single statement
```

Lưu ý: ngõ ra thông thường dùng biến wire, nhưng nếu ngõ ra được gán trong always hoặc initial thì phải khai báo là reg

6. Phát biểu if, case

a. Phát biểu if:

Cú pháp:

```
if (<expression>) true_statement ;
```

```
if (<expression>) true_statement;  
else false_statement;
```

```
if (<expression1>) true_statement1;  
else if (<expression2>) true_statement2;  
else false_statement;
```

Ví dụ: mô tả mux2to1 dùng mô hình hành vi

```
module mux2_1 (i0, i1, s, y);  
    input  i0, i1, s;  
    output y;  
    reg    y;  
    always @ (i0, i1, s) begin  
        if (s == 1'b0)  
            y = i0;  
        else  
            y = i1;  
        end  
    endmodule
```

b. Phát biểu case, casex, casez

Cú pháp:

```
case (biểu thức)  
  giá trị 1: lệnh 1;  
  giá trị 2: lệnh 2;  
  ....  
  giá trị n: lệnh n;  
  default : lệnh 0;  
endcase
```

Ví dụ: mô tả mux 4 to 1 dùng phát biểu case

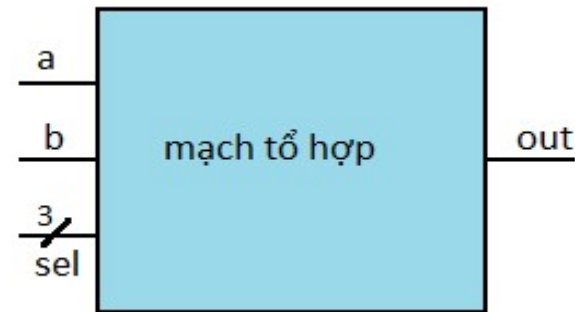
```
Module mux4_1 (out, in, sel);  
  input [3:0] in;  
  input [1:0] sel;  
  output out;  
  reg out;  
  always @(in or sel) begin  
    case (sel)  
      2'b00: out = in[0];  
      2'b01: out = in[1];  
      2'b10: out = in[2];  
      2'b11: out = in[3];  
    endcase  
  end  
endmodule
```

Giống và khác nhau giữa case, casex, casez

- Cú pháp giống nhau
- Trong phát biểu case, biến có giá trị x,z **không** được xem là don't care.
- Trong phát biểu casex, biến có giá trị x,z được xem là don't care.
- Trong phát biểu casez, biến có giá trị z được xem là don't care.

Ví dụ: Mô tả mạch tổ hợp sau dùng cấu trúc case và case x

sel	out
000	0
001	a and b
010	a or b
011	a or b
100	a xor b
101	a xor b
110	a xor b
111	a xor b



sel	out
000	0
001	a and b
01x	a or b
1xx	a xor b

// cấu trúc case

```
module machtohop (a,b,sel, out);  
input a, b;  
input [2:0] sel;  
output out;  
reg out;  
always @ (a,b,sel) begin  
    case (sel)  
        3'b001 : out = a & b;  
        3'b010:  out = a | b;  
        3'b011 : out = a | b;  
        3'b100 : out = a^b;  
        3'b101: out = a^b;  
        3'b110 : out = a^b;  
        3'b111 : out = a^b;  
        default : out = 1'b0;  
    endcase  
end  
endmodule
```

// cấu trúc casex

```
module machtohop (a,b,sel, out);  
input a, b;  
input [2:0] sel;  
output out;  
reg out;  
always @ (a,b,sel) begin  
    casex (sel)  
        3'b001 : out = a & b;  
        3'b01x:  out = a | b;  
        3'b1xx:  out = a^b;  
        default : out = 1'b0;  
    endcase  
end  
endmodule
```

7. phát biểu while, for

Cú pháp:

```
while (condition_expr) statement;
```

Cú pháp:

```
for (init_expr; condition_expr; update_expr) statement;
```

8. Phép gán blocking và non-blocking

a. Phép gán blocking

- Các phát biểu thực thi tuần tự. Phép gán được thực thi ngay khi biểu thức bên phải có kết quả.
- Sử dụng ký hiệu “=” để mô tả phép gán.
- Thứ tự các phát biểu có thể ảnh hưởng đến kết quả cuối cùng
- Thường sử dụng để mô tả mạch tổ hợp

Ví dụ: phép gán blocking

```
always @ (a or b or c)
begin
    t = a | b;
    h = b & c;
    w = ~c;
end
```

Quá trình thực hiện:

Tính toán $(a | b)$ và gán cho t

Tính toán $(b \& c)$ và gán cho h

Tính toán $(\sim c)$ và gán cho w

→ Thích hợp với mô tả mạch tổ hợp

b. Phép gán non-blocking

- Các phát biểu thực thi đồng thời (song song)
- Các phép gán non-blocking chỉ thực hiện gán một lần khi tất cả các biểu thức bên phải đã tính toán xong.
- Sử dụng ký hiệu “ $\leq$ ” để mô tả phép gán.
- Thứ tự các phát biểu không ảnh hưởng đến kết quả cuối cùng
- Thường sử dụng để mô tả mạch tuần tự

Ví dụ: phép gán unblocking

```
always @ (a or b or c)
begin
    t <= a | b;
    h <= b & c;
    w <= ~c;
end
```

Quá trình thực hiện:

Tính toán $(a | b)$ nhưng chưa gán cho t

Tính toán $(b \& c)$ nhưng chưa gán cho h

Tính toán $(\sim c)$ nhưng chưa gán cho w

Gán đồng thời các giá trị mới cho t, h và w

→ Thích hợp với mô tả mạch tuần tự

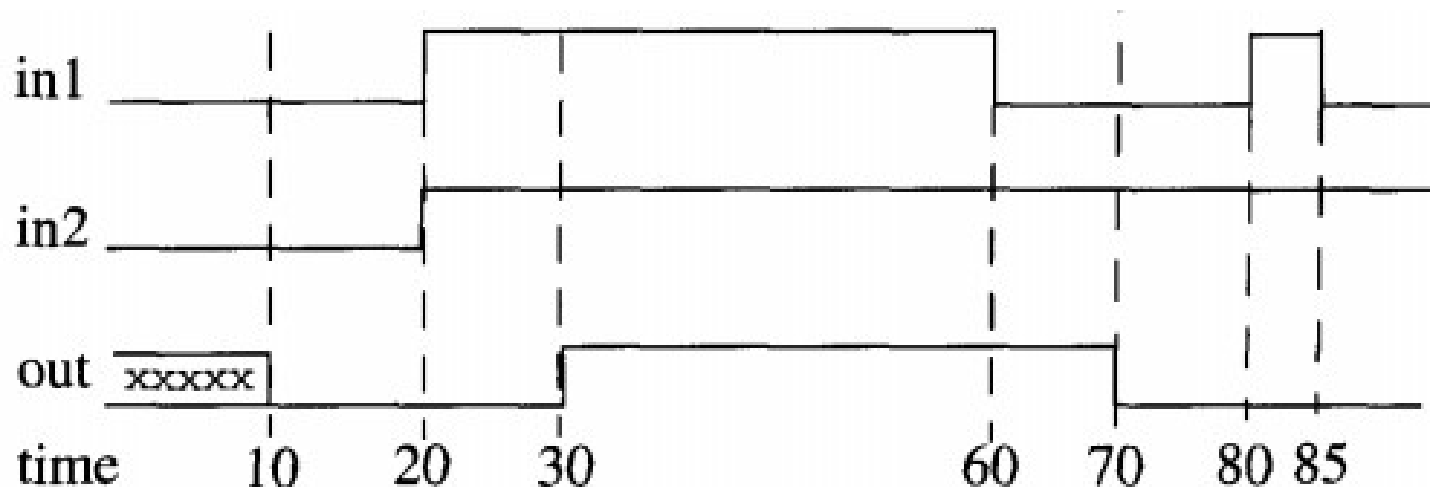
9. Độ trễ của cổng logic (Gate Delays)

- Độ trễ là một giá trị thời gian được gán trên các đường (net) hoặc các cổng để thể hiện sự lan truyền tín hiệu trên các đường hoặc từ các ngõ vào đến ngõ ra của các cổng.
- Trễ lên (rise delay) là độ trễ khi tín hiệu chuyển từ các trạng thái (0, z, x) thành trạng thái “1”.
- Trễ xuống (fall delay) là độ trễ khi tín hiệu chuyển từ từ các trạng thái (1, z, x) thành trạng thái “0”.
- Trễ turn-off (turn-off delay) là độ trễ khi tín hiệu chuyển từ các trạng thái (0, 1, x) thành trạng thái tổng trở cao “z”

a. Delay của phép gán liên tục

Để tạo độ trì hoãn tính từ khi giá trị của một toán hạng bên phải phép gán thay đổi cho đến khi giá trị net bên tay trái được cập nhật sự thay đổi giá trị đó

VD : assign #10 out = in1 & in2;



b. Cú pháp:

(value1, value2, value3)

trong đó: value1 : rise delay
 value2 : fall delay
 value3 : turn-off delay

➤ Nếu value3 = 0 thì cú pháp là:

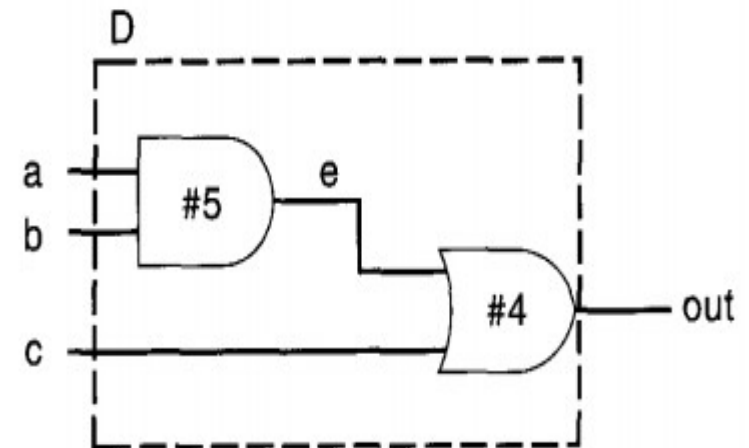
(value1, value2)

➤ Nếu value1 = value2 = value3 thì cú pháp là:

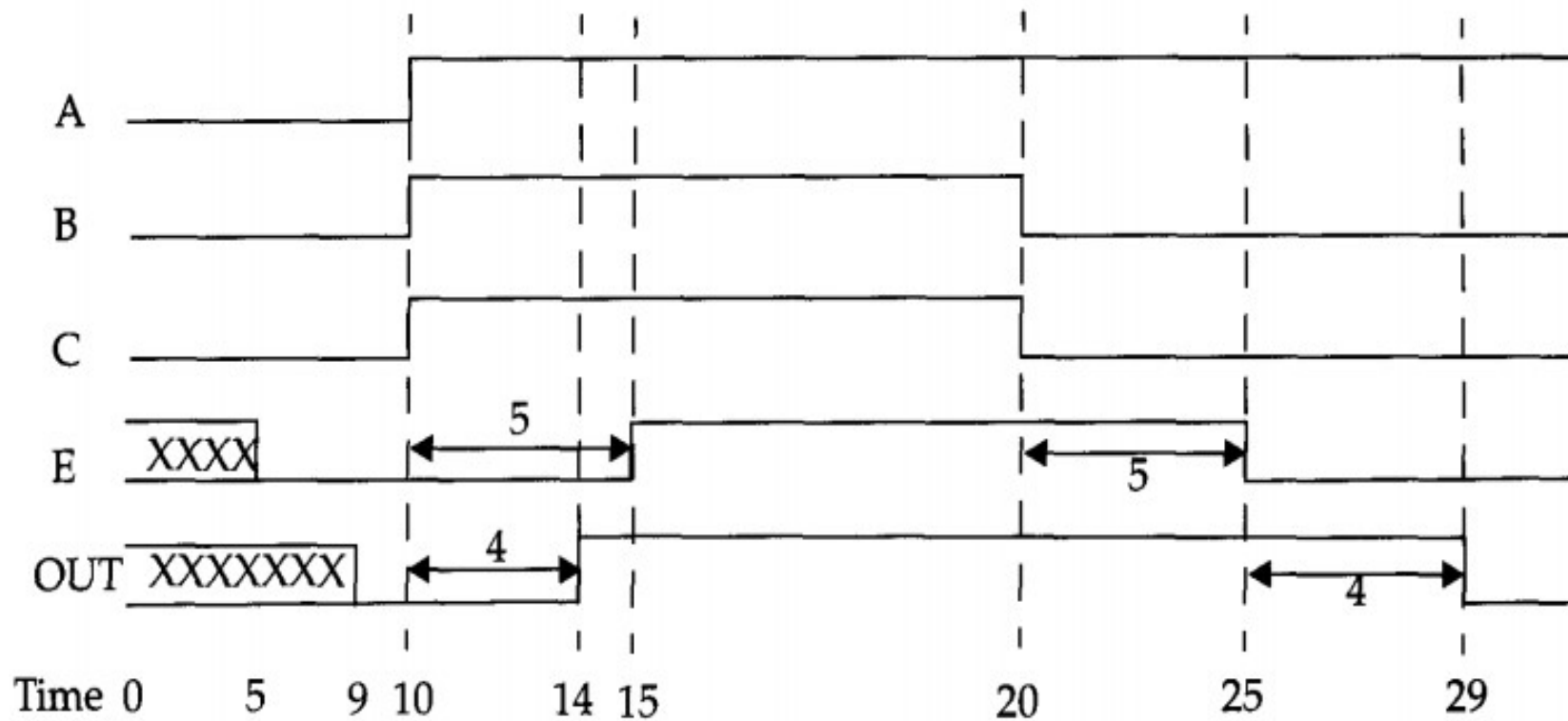
(value) hoặc # value

Ví dụ 3: viết chương trình mô tả mạch logic sau

```
Module    delay (out, a, b, c);  
    output    out;  
    input     a, b, c;  
    wire      e;  
    //tạo trẽ cho cổng and và or  
    and # (5) and1 (e, a, b);  
    or  # (4) or1 (out, e, c);  
endmodule
```



Kết quả mô phỏng



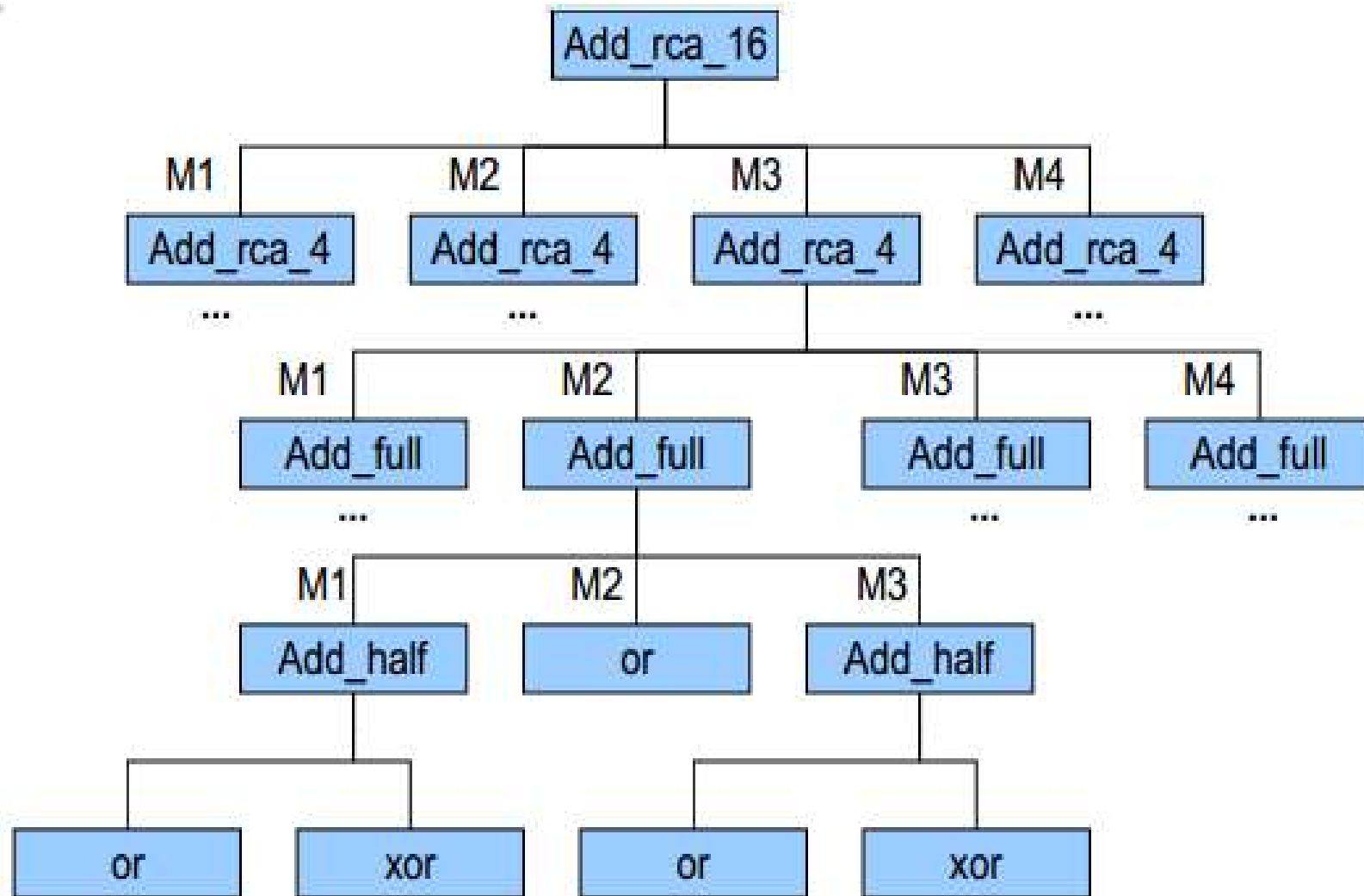
III. Thiết kế từ trên xuống (top-down)

- Verilog hỗ trợ cấu trúc phân cấp bằng cách cho phép modules được nhúng trong modules khác.
- Modules cấp độ cao hơn tạo thể hiện của module ở cấp độ thấp và giao tiếp với chúng thông qua các đầu vào, đầu ra và đầu vào ra hai chiều.
- Cấu trúc phân cấp giúp người thiết kế chia một hệ thống thiết kế ra thành các module nhỏ hơn để dễ thiết kế và kiểm soát luồng dữ liệu trong quá trình thiết kế.

a. Nguyên tắc thiết kế top-down

- top-level module là module ở cấp cao nhất
- module thấp nhất chứa các hàm được định nghĩa sẵn trong verilog
- các module được viết trong một hoặc nhiều file khác nhau
- module cấp cao muốn gọi module cấp thấp hơn thì trong chương trình của nó phải có lệnh gọi module thấp đó

VD: Cấu trúc phân cấp của bộ cộng 16 bit



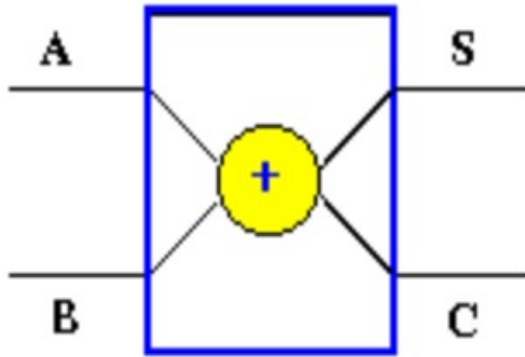
b. Module instantiation

- Là module được định nghĩa lồng trong một module khác
- Lệnh gọi và gán đặc tính cho module instantiation sẽ tạo ra một hoặc nhiều bản sao của module được định nghĩa
- Cách gọi module instantiation như sau:

```
module_name  instance_name (port_connection_list)
```

Ví dụ : viết code Verilog mô tả mạch FA dùng HA

Mạch cộng HA



Chức năng: HA cộng hai số nhị phân A và B một bit không có nhớ

Biểu thức ngõ ra:

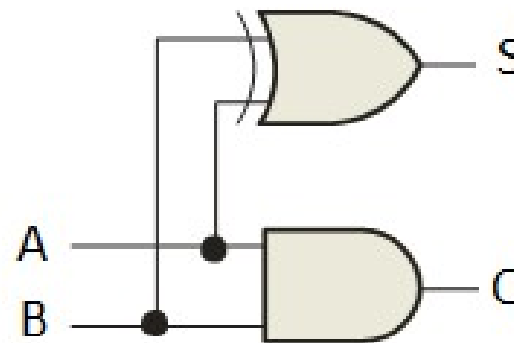
$$S = A \oplus B$$

$$C = A . B$$

Bảng trạng thái:

Vào		Ra	
A	B	S	C _o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sơ đồ mạch HA:



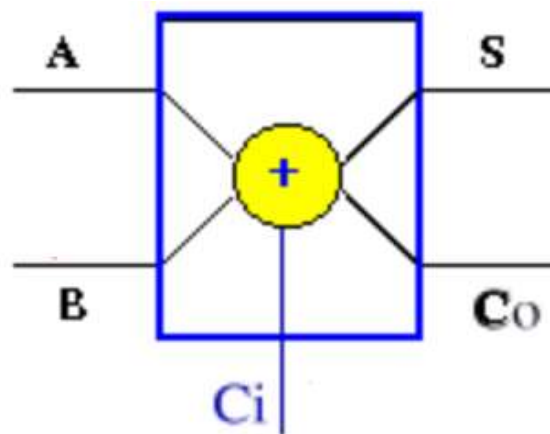
Mạch cộng FA

Chức năng:

FA cộng hai số nhị phân A và B một bit có nhớ.

Bảng trạng thái:

Vào			Ra	
A	B	C_i	S	C_o
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1



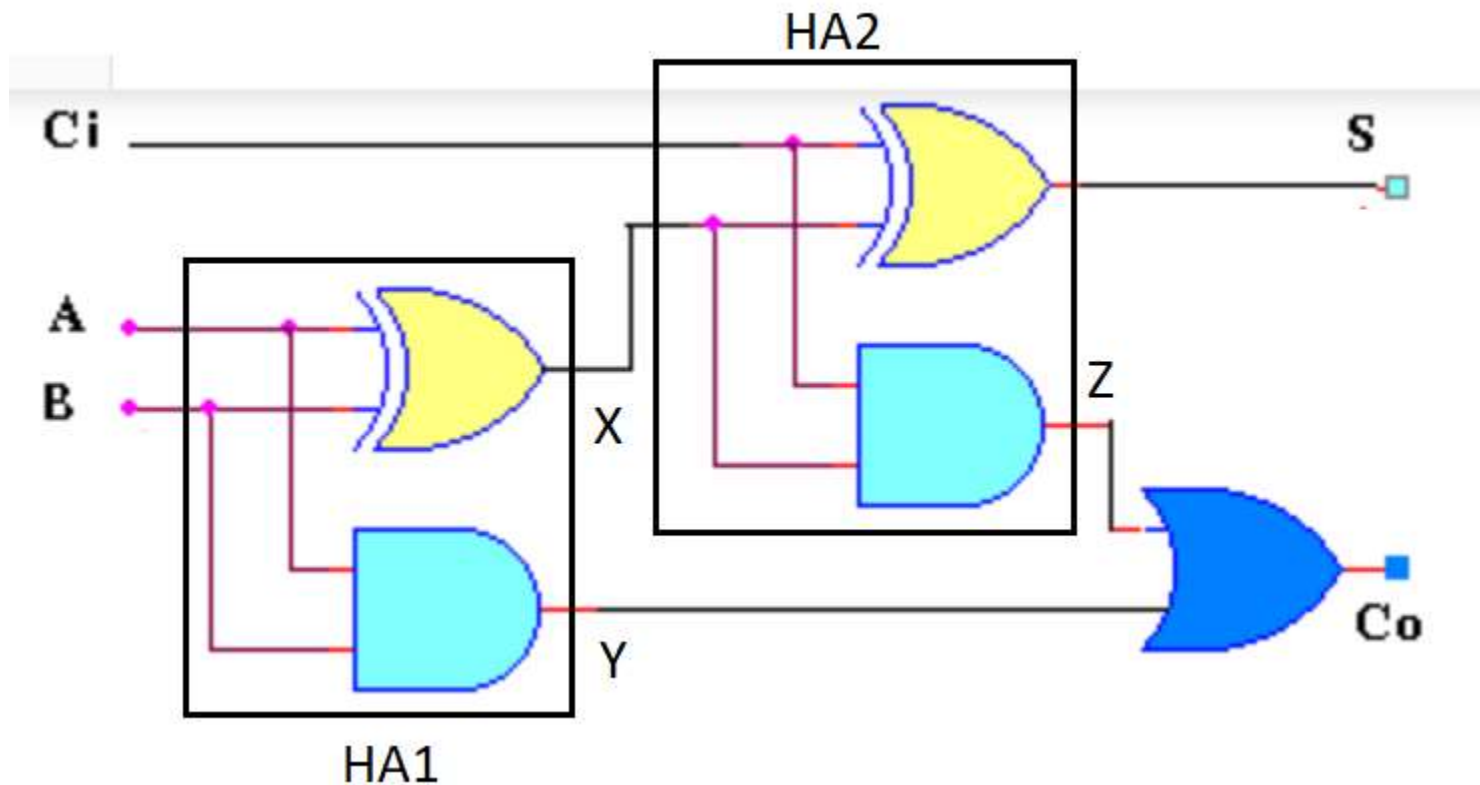
Biểu thức ngõ ra:

$$S = (A \oplus B) \oplus C_i$$

$$C_o = AB + (A \oplus B)C_i$$

Bài tập 1: viết code Verilog mô tả mạch FA dùng HA

Mạch cộng FA dùng HA



IV. Các mô hình viết code trên Verilog HDL

- Mô hình cấu trúc: chủ yếu sử dụng các hàm logic not, and, or... được hỗ trợ trên verilog. Thích hợp mô tả các mạch logic có số lượng cổng nhỏ và có sẵn sơ đồ mạch logic
- Mô hình luồng dữ liệu: chủ yếu sử dụng phép gán assign và mức độ ưu tiên của các toán tử trong biểu thức . Thích hợp mô tả các mạch logic có sẵn biểu thức ngõ ra, số lượng input/output vừa phải
- Mô hình hành vi: chủ yếu sử dụng always(...) là điều kiện để mạch hoạt động. Dùng mô tả mạch logic khi biết nó hoạt động thế nào chứ ko cần sơ đồ mạch hay biểu thức ngõ ra

Lưu ý:

Mô hình cấu trúc, luồng dữ liệu thường dùng mô tả mạch tổ hợp nhỏ và vừa

Mô hình hành vi thường dùng mô tả mạch tuần tự, mạch tổ hợp kích thước lớn

Mô phỏng tổng hợp và Test bench

a. Mô phỏng tổng hợp

mô phỏng chức năng thiết kế trực tiếp trên RTL code đã viết đúng syntax của ngôn ngữ VHDL hoặc Verilog. Mô phỏng mức RTL không tính đến timing (độ trễ trên các cổng hay các đường kết nối)

b. Test bench

Test bench là một công cụ mô phỏng, cho phép ta giả lập các tác động ở ngõ vào và mô phỏng phản ứng của hệ FPGA, từ đó đánh giá xem thiết kế đã đạt yêu cầu chưa.



Thank you for
listening

